JAM2JAM XO: AN EDUCATIONAL GENERATIVE MUSIC ACTIVITY FOR THE OLPC XO

Thorin Kerr

School of Music,
The University of Queensland

Andrew R. Brown

Queensland Conservatorium of
Music, Griffith University

ABSTRACT

Jam2jam XO is a software application developed for the One Laptop Per Child (OLPC) 'XO' laptop, an educational computer designed for use by children in poor or developing countries.

Jam2jam XO is one of family of jam2jam educational multimedia software instruments. While jam2jam XO adopts techniques used by the other jam2jam's, jam2jam XO has been designed and developed to take particular advantage of the resources available on the XO laptop. The XO laptop boasts a number of innovative features, including wireless mesh networking and the 'Sugar' interface.

The jam2jam project holds a philosophy around teaching and education that shares much in common with principals held by the OLPC initiative. By developing a version of the jam2jam software for the OLPC XO laptop, jam2jam XO looks to further the objectives of both the jam2jam project and the OLPC initiative.

Having achieved over 30 000 downloads, jam2jam XO continues to be a popular activity used throughout the OLPC community.

1. ABOUT THE OLPC XO

The One Laptop Per Child (OLPC) initiative, created by Nicholas Negroponte at the Massachusetts Institute of Technology, aims to provide children in poor and developing countries with a rugged, low-cost, low-power, network-connected laptop. [1,8] To date approximately 2.5 million laptops have been distributed to children throughout the world in many places including Uruguay, Nepal, and Mongolia. Rural communities in Australia have also received laptops distributed by OLPC Australia [2].

The OLPC XO laptop is a low cost, rugged, low-powered laptop with built-in WiFi mesh networking, long lasting battery, USB ports, and a screen that is visible in daylight hours.

The Sugar Learning Platform is the 'interface' for the XO laptop. Sugar sits somewhere between a desktop and an operating system. The Sugar interface is the means by which a user interacts with applications - called Activities - saved data, and other network connected XO laptops.

Interaction with Sugar is quite different to interacting with a conventional operating system such as Windows, Mac, or even Linux: There are for example no files or folders in Sugar. Instead, snapshots of activity sessions are stored in a single location called a Journal.

Connecting to other XO laptops is done simply by 'sharing' an activity with other XO laptops in the 'neighbourhood'. The XO laptop can establish the connection using a variety of methods, either through a shared server, with the aid of a wireless router or directly with other XO laptops using peer to peer wireless mesh or ad hoc networking.

For a developer, Sugar offers an API to build a GUI, access devices and manage network connections. Sugar is built on a cut down version of Fedora Linux, and so many Linux resources are also available. The Sugar API is written in the Python programming language, and many of the additional libraries and toolkits provide Python interfaces [3]. Sugar activities are typically written in Python, although other programming languages can be used.

A Core Principal of the OLPC initiative is to be Free and Open Source [4]. As such all Sugar activities are released under an open source license. Jam2jam XO is released under the version 2 of the GNU General Public License.

2. ABOUT JAM2JAM XO

Jam2jam XO development began in 2009 to 2010 with the support of the Australasian Co-operative Research Centre for Interaction Design [5]. Jam2jam XO currently exists as a private project of the author.

The jam2jam suite of applications forms part of the Network Jamming project, led by Steve Dillon and Andrew Brown. All the jam2jam variants act as educative collaborative multi-media tools for people to create music and/or imagery. The software aims to enhance engagement and learning by being fun, and providing social interaction [6].

2.1. jam2jam AV

Jam2jam XO most closely resembles jam2jam AV. Built in the Impromptu live-coding environment, running on Mac OSX, jam2jam AV offers parameter control over generative musical transformations. The jam2jam AV user interface offers free movement of an instrument icon in a two dimensional space. The horizontal and vertical movement correspondingly adjusts two musical parameters being performed by the

instrument. Different instruments can be selected, resulting in different musical transformations for each instrument.

Jam2jam AV communicates with other jam2jam AV's on a local network or over the internet, so-as to allow virtual 'bands' to collaborate. The audible result of the collaboration is duplicated in each connected jam2jam AV, i.e. the music sounds the same on each computer [7].

Jam2jam AV also provided video manipulation, with pre-recorded video's, still images, or live video from a camera able to be processed. It is worth noting that jam2jam AV can consume a considerable amount of CPU and memory resources on a modern Apple computer.

2.2. Using jam2jam XO

The jam2jam XO 'Activity' runs on the Sugar Learning Platform for use on the XO Laptop. Jam2jam XO borrows some concepts from jam2jam AV, most notably the two dimensional parameter control. However the implementation of these concepts is significantly different. In part the differences are needed to accommodate differences in hardware, computing power and software resources between a modern Apple computer and the OLPC XO [8]. As jam2jam XO was also a complete re-write of the jam2jam software, the opportunity was taken to re-think much of the user interaction in this environment.

In keeping with OLPC's principal of collaboration, jam2jam XO is most effective as a networked activity. When another player joins a shared jam2jam XO activity, each player gains control of an instrument. i.e. one performer may operate the guitar, while the other performer operates the drums. Audio for the entire musical performance is shared across the XO laptops, such that one performer will hear only the guitar audio track from their XO laptop, while the original player operates the drums and hears all the remaining instruments from their XO. In this way, a 'band' forms with each player contributing to the audible result. The band is limited to four performers, with each player taking on a unique instrument.

Visual effects are relatively limited, as image and video processing can easily interrupt music playback on the XO laptop. The XO laptop camera is used to take pictures of the user. An accumulation of captured images is cycled through in time with the beat of the music.

3. INSIDE JAM2JAM XO

Jam2jam XO is written almost entirely in the Python programming language. There are a number of Python interfaces employed to access other tools included in the Sugar environment: The user interface makes use of a mixture of PyGTK – Python wrappers over the GTK GUI toolkit, and the 'olpcgames' library – itself a wrapper for Pygame – a 2D game and multimedia library. The olpcgames library also offers access to XO networking and the camera.

The musical styles are stored as MIDI files. These files are read, and the notes modified by algorithms, before being sent to a synthesiser.

The synthesiser in use is Csound, which is also a standard inclusion in the Sugar environment. Csound runs in real-time, with the 'notes', sent directly from the Python environment through the Python bindings of the Csound API [9]. Jam2jam XO generates appropriate Csound orchestra code at launch time, based on which MIDI files it finds in its activity bundle. This allows the addition, or change of musical styles to jam2jam XO as simple including new MIDI files into the activity bundle.

3.1. Timing

Timing of notes using the standard Python modules was found to be too inaccurate for musical purposes. The Pygame environment offers an accurate timing option, however the increase in CPU usage and the relatively slow frame rate also proved limited for musical use. The Csound API however offers a callback function which can repeatedly execute a Python function at the Csound control rate. This is fast enough for accurate musical timing. Note that Csound itself runs in its own thread, uninterrupted by Python threading or interpreter locking issues.

Jam2jam XO uses a scheduling model loosely based on the scheduling used in the Impromptu live-coding environment [7]. Events (such as notes to be sent to Csound) are sent to a scheduler queue with an execution time specified in the future. The callback function, running at the Csound control rate, polls the queue for pending events, checks the scheduled execution time with the current time and executes anything due or past due. When the event is executed it is removed from the queue.

3.2. Synchronising XO's

When a jam2jam XO activity is shared across a network to form a band, jam2jam XO attempts to play all the instruments together in time. On a fast connection a fixed latency calculation might be good enough. However on the low powered XO, with a possibly poor network connection, running a non-real-time operating system with multiple threads in a higher level interpreted programming language, the synchronisation task requires more care.

jam2jam XO attempts synchronisation using a model resembling Simple Network Time Protocol with some added sanity measures to accommodate erratic timing messages [10].

One jam2jam XO becomes the reference clock. It periodically broadcasts a message about where it is up to in the music to the other connected 'band members'. When the band members receive this message they align themselves to the same point in the music.

There is typically some delay in the time it takes the reference jam2jam XO to broadcast a message to the time the other jam2jam's receive it. This delay is calculated by the sending of a time-stamped round-trip message to tvia the reference jam2jam XO. The latency of the broadcast time message is assumed to be half the time it took the round trip message to return.

The accuracy of the latency calculation and its correspondence to the broadcast message delay can be adversely affected by numerous factors which have been mentioned above. This situation is somewhat mitigated by calculating the average of an accumulation of recorded delay calculations. The delay messages are also filtered so-as to ignore rogue results which would otherwise skew the data.

Despite these efforts, a poor or variable network connection, or timing errors in the reference jam2jam, will still cause the music to get out of time. In most cases however timing errors which do creep in eventually correct themselves.

4. MUSICAL TRANSFORMATIONS

Musical transformations occur by moving an instrument either horizontally or vertically in the main interface. When a parameter is selected for transformation along an axis, the user can control the amount of transformation that is applied to that parameter.

The transformations used in jam2jam XO are a mixture of audio modifications as well as changes in the performance of musical notes. The transformations employed are; pitch stretching/compression, rhythmic density, articulation, volume, and timbre. A brief explanation of these transformations is provided below. These methods are described in more detail in the paper 'Adaptive Music Techniques' by Brown and Kerr [11].

4.1. Pitch stretching/compression

An increase in pitch parameter has the effect of expanding the range of pitches performed in an instrumental part. Thus a melody line will have a larger pitch contour, with greater interval sizes. Chordal parts will have larger steps chosen to describe the chord.

Conversely, a decrease in the pitch parameter will reduce the pitch range, such that at the minimum level, all pitches will centre on a single note.

The drum section represents a special case. In jam2jam XO, an increase in pitch effectively filters out lower drum hits (bass drums and toms for example), while a reduction in pitch will filter out high pitched percussion. A side effect of this approach is a reduction in rhythmic density toward the extremes, with many drum ‘hits’ not being performed.

4.2. Density

The approach to modifying rhythmic density differs in jam2jam XO when compared to jam2jam AV. In the latter case three versions of a musical track (recorded as MIDI files), each representing differing levels of rhythmic density are mixed.

In jam2jam XO only a single musical track is used. At maximum density the music is played unaltered. A reduction in the rhythmic density filters notes out of the performance. The selection of which notes are removed is based on a hierarchical interpretation on rhythmic importance in a 'beat' or bar.

Reduction in rhythmic density does not lead to silent gaps in the music. Instead, the durations of the remaining notes are lengthened so-as to occupy the space left by the removed notes.

4.3. Articulation

Articulation is implemented as a duration multiplier on each note. This has the effect of increasing or reducing the duration of note lengths. Overlapping notes are allowed in jam2jam XO, with the exception of the Bass line, which is implemented as a monophonic instrument.

4.4. Volume, Timbre and Tempo

The volume and timbre parameters perform simple audio transformations. Volume adjustment literally changes the output level of an instrument. Timbre modification passes the signal through a low pass filter. Because the frequency response of the XO built in speakers is poor, and frequencies less than 200hz become inaudible, some distortion is applied to the signal as the cutoff frequency gets lower. This maintains the presence of higher partials in the sound.

At this stage tempo modification has not been implemented in jam2jam XO, although it is planned to be included in future releases. An approach to tempo change that maintains synchronisation between XO's in a network situation is being tested.

5. THE IMPACT OF JAM2JAM XO

Jam2jam has been a popular activity on the XO. At the time of writing it had been downloaded 34,600 times. In a single day on April 8th, 2010 it received 417 downloads, and for some time was the most popular 'Media' activity downloaded at Sugar labs.

Information on its actual use is harder to assess. A single download can be further distributed throughout a school from an 'XO school server', suggesting there could be many more users than downloads.

Direct feedback from end users is uncommon, as there are numerous layers of support available in the Sugar/OLPC community, including teachers, OLPC country deployments and the wider OLPC and Sugar community.

Instead, requests from 'higher up the tree' are more common. For example, there have been documentation and technical queries from the Plan Ceibal initiative in

Uruguay, responsible for managing half a million OLPC XO's.

The open source nature of Sugar and Sugar Activities also means that many technical issues are resolved without the need for the original developer to be approached. Modified versions of jam2jam XO have appeared with contributions from other developers to fix platform specific issues. A modified version of jam2jam XO was included in a recent Australian build of the 'XO server' (XS) software.

6. THE FUTURE OF JAM2JAM XO

Jam2jam aims to be an engaging, useable and creative activity for both children and teachers. In order to ascertain whether these aims are being met a form of monitoring and evaluation is required. To conduct some form of monitoring, avenues of obtaining more direct feedback from actual users are currently being explored.

Jam2jam will increasingly adhere to OLPC Core Principals, and in particular the two 'extra' principals created by OLPC Australia [12]. One of these extra OLPC Australia Core Principals, which is also a cornerstone of the jam2jam philosophy, is that of Community Engagement [4]. As such, jam2jam will be looking to expand its presence throughout Australia.

The other OLPC Australia principal is concerned with 'Empowering Teachers'. To that end, jam2jam XO will incorporate services to teachers. Jam2jam AV also provided features of this nature, such as offering time-stamped data on actual usage by students, allowing learning outcomes to be measured.

The jam2jam approach to software development is to implement features based on user feedback, and so actual development outcomes will rely on the efficacy of jam2jam XO as it is used in real world situations.

jam2jam XO has recently been recognised by the OLPC contributors program. As a result the developer has obtained two XO 1.5's to continue development.

7. REFERENCES

[1] Negroponte, N. “Nickolas Negroponte,” http://web.media.mit.edu/~nicholas/, accessed 20 June, 2012.

[2] OLPC, “Deployments,” http://wiki.laptop.org/go/Deployments, accessed 20 June, 2012.

[3] Sugarlabs, “Sugar System Stack,” http://wiki.sugarlabs.org/go/Sugar_System_Stack, accessed 20 June, 2012.

[4] OLPC, “, ” “OLPC:Five principles,” http://wiki.laptop.org/go/OLPC:Five_principles, accessed 20 June, 2012.

[5] ACID, “jam2jam XO: collaborative media performance software for OLPC, ” http://acid.net.au/indexea6e.html?option=com_content&task=view&id=147&Itemid=218, accessed 20 June, 2012.

[6] Brown, A. R., Dillon, S., Kerr, T. and Sorensen, A., “Evolving Interactions: Agile design for networked media performance” in *OZCHI 2009 Proceedings : Design: Open 24/7*, Melbourne, 2009, pp. 41-48.

[7] Sorensen, A.”Impromptu: An interactive programming environment for composition and performance.” *Proceedings of the Australasian Computer Music Conference*, Brisbane, 2005, pp. 149-153.

[8] OLPC, “Hardware Specification,” http://wiki.laptop.org/go/Hardware_specification

[9] Gogins, M. *"Csound and CsoundVST API Reference Manual”*, 2006, http://csound.sourceforge.net/refman.pdf

[10] NTP: The Network Time Protocol, http://www.ntp.org/, accessed 20 June, 2012.

[11] Brown, A. R. and Kerr, T. “Adaptive Music Techniques”. *Improvise: The Australasian Computer Music Conference,* Brisbane, Australia: pp. 26-31.

[12] OLPC Australia, “Core Principles,” http://edu.laptop.org.au/vision/core-principles, accessed 20 June, 2012.